

Total Variation Image Denoising with Gradient Descent and a Newton-Type Lagged Diffusivity Method

Kemal Sivro, Student ID 20015

Faculty of Electrical Engineering, University of Sarajevo, Bosnia and Herzegovina

Seminar paper in Numerical Optimisations, Data Science and Artificial Intelligence

Professor: Izudin Džafić, academic year 2025/26

Source code: <https://github.com/vrosi21/TV-Image-Denoising>

Abstract—Total variation denoising recovers an image by minimising the sum of a quadratic data term and a weighted total variation of the image. This paper compares two solvers for a smoothed version of this model, both implemented in C++ on the `natID` framework and run on identical seeded noise: gradient descent with Armijo backtracking, and a Newton-type method that solves one sparse linear system per iteration, known as the lagged diffusivity or iteratively reweighted least squares method. On three 225×225 test images with noise level $\sigma = 0.1$ and $\lambda = 0.1$, both methods reach the same image quality. Gradient descent costs 3.5 to 5.3 ms per iteration and reaches its best peak signal-to-noise ratio (PSNR) within 4 to 40 iterations; the Newton-type method costs 101 to 123 ms per iteration, reaches its best PSNR within 2 to 5 iterations and reduces the relative change of the iterate by a factor of ten or more every ten iterations. Its smoothing parameter introduces a bias: its limit has a 0.9 to 1.7% higher exact TV energy than gradient descent after 300 iterations. At the PSNR-optimal λ both methods agree within 0.08 dB. A scaling study shows that the dense Newton system grows like N^3 and reaches 1.6 s per iteration at $N = 4096$ pixels, while the sparse $LDL^\top$ path grows almost linearly and needs 0.5 s at $N = 202,500$.

Index Terms—total variation, image denoising, gradient descent, Armijo line search, Newton method, lagged diffusivity, sparse Cholesky

I. INTRODUCTION

Noise removal that preserves edges is a basic task in image processing. The model of Rudin, Osher and Fatemi [1] achieves this by penalising the total variation of the image: smooth noise-like oscillations have a large total variation, while a sharp edge of a given height costs no more than a gradual ramp. The resulting problem is convex, so it has a unique minimiser, but the total variation is not differentiable where the image gradient vanishes. Practical methods either smooth the functional [2], work with a dual formulation [3] or use primal-dual splitting [4], [5].

This work studies the smoothed approach, because it allows a direct comparison of a first-order and a second-order method on the same objective. The contributions are:

- an implementation of gradient descent with Armijo backtracking and of the lagged diffusivity Newton-type method, with a dense and a sparse linear solver;
- a comparison on three standard images in terms of PSNR, exact TV energy, relative change and wall time, together with studies of the regularisation weight λ , the smoothing parameter ε and the noise level;
- a scaling study of the cost per iteration from 64 to 202,500 pixels;
- a cross-platform desktop application that runs every experiment interactively.

Section II states the model, Section III the two methods and Section IV the implementation. Results follow in Section V and are discussed in Section VI.

II. MODEL

A. Discrete total variation

Let $f \in [0, 1]^{H \times W}$ be the observed image and $N = HW$. Forward differences with a zero difference at the last column and row define

$$(D_x u)_{i,j} = \begin{cases} u_{i,j+1} - u_{i,j}, & j < W, \\ 0, & j = W, \end{cases} \quad (1)$$

$$(D_y u)_{i,j} = \begin{cases} u_{i+1,j} - u_{i,j}, & i < H, \\ 0, & i = H, \end{cases} \quad (2)$$

and the discrete gradient $D = \begin{bmatrix} D_x \\ D_y \end{bmatrix} \in \mathbb{R}^{2N \times N}$. The isotropic total variation and its smoothed version are

$$\text{TV}(u) = \sum_{i,j} \sqrt{(D_x u)_{i,j}^2 + (D_y u)_{i,j}^2}, \quad (3)$$

$$\text{TV}_\varepsilon(u) = \sum_{i,j} \sqrt{(D_x u)_{i,j}^2 + (D_y u)_{i,j}^2 + \varepsilon^2}, \quad \varepsilon > 0. \quad (4)$$

The function $\sqrt{t^2 + \varepsilon^2}$ behaves like $|t|$ for $|t| \gg \varepsilon$ and like a quadratic near zero, similar to the Huber function [6]; the application therefore calls it Huber smoothing.

B. Objective

The denoised image minimises

$$F_\varepsilon(u) = \frac{1}{2}\|u - f\|_2^2 + \lambda \text{TV}_\varepsilon(u), \quad (5)$$

where $\lambda > 0$ balances noise removal against fidelity. Because the data term is 1-strongly convex and TV_ε is convex, F_ε has a unique minimiser for every $\varepsilon \geq 0$. The solvers minimise F_ε , but for a fair comparison all reported energies use the exact functional $E(u) = \frac{1}{2}\|u - f\|_2^2 + \lambda \text{TV}(u)$, i.e. $\varepsilon = 0$. Quality is measured against the clean image u° by

$$\text{PSNR}(u) = 10 \log_{10} \frac{1}{\text{MSE}}, \quad \text{MSE} = \frac{1}{N} \|u - u^\circ\|_2^2. \quad (6)$$

C. Derivatives

With the per-pixel weights

$$w_{i,j}(u) = \left((D_x u)_{i,j}^2 + (D_y u)_{i,j}^2 + \varepsilon^2 \right)^{-1/2} \quad (7)$$

and $W(u) = \text{diag}(w, w) \in \mathbb{R}^{2N \times 2N}$, the gradient is

$$\nabla F_\varepsilon(u) = u - f + \lambda D^\top W(u) D u. \quad (8)$$

Written per pixel, with $d^x = D_x u$ and $d^y = D_y u$,

$$\begin{aligned} [D^\top W D u]_{i,j} = & -w_{i,j} (d_{i,j}^x + d_{i,j}^y) \\ & + w_{i,j-1} d_{i,j-1}^x + w_{i-1,j} d_{i-1,j}^y, \end{aligned} \quad (9)$$

with terms outside the image set to zero. This is the negative discrete divergence of $\nabla u / |\nabla u|_\varepsilon$.

Differentiating once more gives the Hessian

$$\nabla^2 F_\varepsilon(u) = I + \lambda D^\top B(u) D, \quad (10)$$

where $B(u)$ consists of one 2×2 block per pixel,

$$B_{i,j} = \frac{1}{r} I_2 - \frac{1}{r^3} \begin{bmatrix} d^x \\ d^y \end{bmatrix} \begin{bmatrix} d^x & d^y \end{bmatrix}, \quad r = \frac{1}{w_{i,j}}. \quad (11)$$

The eigenvalues of $B_{i,j}$ are $1/r$ and ε^2/r^3 , so $0 \prec B(u) \preceq W(u)$ and

$$I \preceq \nabla^2 F_\varepsilon(u) \preceq A(u) = I + \lambda D^\top W(u) D. \quad (12)$$

Since $\|D\|_2^2 \leq 8$ and $w \leq 1/\varepsilon$, the gradient (8) is Lipschitz continuous with constant

$$L \leq 1 + \frac{8\lambda}{\varepsilon}, \quad (13)$$

so the condition number of F_ε can be as large as $1 + 8\lambda/\varepsilon$.

III. METHODS

A. Gradient descent with Armijo backtracking

Each iteration computes $g_k = \nabla F_\varepsilon(u_k)$ and finds a step length t by backtracking from $t = 1$ with factor $\beta = \frac{1}{2}$ until the Armijo condition [7]

$$F_\varepsilon(u_k - t g_k) \leq F_\varepsilon(u_k) - \alpha t \|g_k\|_2^2, \quad \alpha = 0.01, \quad (14)$$

holds, with at most 40 reductions. The update is projected onto the valid pixel range,

$$u_{k+1} = \text{clip}(u_k - t g_k, 0, 1). \quad (15)$$

Algorithm 1 Lagged diffusivity (Newton-type) iteration

Input: noisy image f , λ , ε , iterations K

```

1:  $u \leftarrow f$ 
2: for  $k = 1, \dots, K$  do
3:   compute  $w_{i,j}(u)$  by (7)
4:   assemble  $A = I + \lambda D^\top W D \triangleright$  five nonzeros per row
5:   if  $N \leq 4096$  then solve  $A u_{\text{new}} = f$  by dense LU
6:   else solve  $A u_{\text{new}} = f$  by sparse  $LDL^\top$ 
7:   end if
8:    $u \leftarrow \text{clip}(u_{\text{new}}, 0, 1)$ 
9: end for
```

The implementation fixes $\varepsilon = 10^{-3}$ for gradient descent. With (13), $\lambda = 0.1$ gives $L \leq 801$, and backtracking accepts steps of order $1/L$. Convergence is linear with a rate that depends on L [8], [9]: components of the error along directions of large curvature, such as pixel-scale oscillations, are removed in a few iterations, while smooth components decay slowly.

B. Newton-type lagged diffusivity method

A full Newton step would use (10). The lagged diffusivity method [2] instead freezes the weights at the current iterate and uses the matrix $A(u_k)$ from (12). Because (8) can be written as $\nabla F_\varepsilon(u) = A(u) u - f$, the Newton-type step with this matrix is

$$u_{k+1} = u_k - A(u_k)^{-1} \nabla F_\varepsilon(u_k) = A(u_k)^{-1} f, \quad (16)$$

so each iteration solves one symmetric positive definite system

$$(I + \lambda D^\top W(u_k) D) u_{k+1} = f. \quad (17)$$

This is also an iteratively reweighted least squares (IRLS) step, since u_{k+1} minimises the weighted quadratic $\frac{1}{2}\|u - f\|_2^2 + \frac{\lambda}{2} \sum w_{i,j}(u_k) |(Du)_{i,j}|^2$.

The iteration decreases F_ε monotonically. The function $s \mapsto \sqrt{s + \varepsilon^2}$ is concave, so for every pixel $\sqrt{s + \varepsilon^2} \leq \sqrt{s_0 + \varepsilon^2} + \frac{w_0}{2}(s - s_0)$. Summing over pixels shows that

$$G(v; u) = \frac{1}{2} \|v - f\|_2^2 + \frac{\lambda}{2} \sum_{i,j} w_{i,j}(u) |(Dv)_{i,j}|^2 + c(u) \quad (18)$$

satisfies $G(v; u) \geq F_\varepsilon(v)$ and $G(u; u) = F_\varepsilon(u)$. The step (16) minimises $G(\cdot; u_k)$, hence $F_\varepsilon(u_{k+1}) \leq G(u_{k+1}; u_k) \leq G(u_k; u_k) = F_\varepsilon(u_k)$. Global convergence of the method was proved by Chan and Mulet [10]. Its rate is linear, not quadratic, because the curvature term in (11) is ignored; primal-dual Newton methods recover it [11]. The implementation clips u_{k+1} to $[0, 1]$ as in (15) and exposes ε to the user, with default 10^{-2} .

C. Linear algebra

In row-major pixel order $A(u)$ has at most five nonzeros per row:

$$A_{k,k} = 1 + \lambda(w_k[j < W] + w_{k-1}[j > 1] + w_k[i < H] + w_{k-W}[i > 1]), \quad (19)$$

$$A_{k,k-1} = -\lambda w_{k-1}, \quad A_{k,k-W} = -\lambda w_{k-W}, \quad (20)$$

where k is the index of pixel (i, j) and $[\cdot]$ is 1 when the condition holds. For $N \leq 4096$ the system is assembled as a dense matrix and solved with the LU solver of the natID matrix library, which costs $O(N^2)$ memory and $O(N^3)$ time. A dense matrix for a 225×225 image would need $N^2 = 2.6 \times 10^9$ entries, about 20 GB, so larger images use the sparse $LDL^\top$ solver of natID with its own fill-reducing ordering. For regular grids, nested dissection orderings bound the fill to $O(N \log N)$ and the factorisation work to $O(N^{3/2})$ [12], [13].

IV. IMPLEMENTATION

The application is written in C++20 with the natID framework [14], which provides the user interface, the dense and sparse matrix libraries and the installer tooling. A GitHub Actions pipeline builds installers for Windows, macOS on Apple Silicon and Intel, and Linux; the code contains no platform-specific calls.

A. Design

The numerical classes do not depend on the user interface. ImageData stores a grayscale image as 32-bit floating point values in $[0, 1]$, loads PNG and JPEG files through natID and adds reproducible noise $f = \text{clip}(u^\circ + \sigma z, 0, 1)$, where z comes from the Mersenne Twister with a hand-written Box-Muller transform, so a seed gives identical noise on every platform. Both solvers implement the interface ITVSolver, whose denoise method accepts an observer called after every iteration; the observer records metrics and delivers cancellation, so the solvers contain no measurement or threading code. DenoisingMetrics computes $E(u)$, PSNR and the relative change, and DenoisingRunner executes a comparison or a λ study without any user interface, on a background thread in the application and from a console program in this paper.

B. Application

Fig. 1 shows the main window. The sidebar selects the image and holds sliders for σ , λ , ε and the number of iterations, the seed, the methods to run and the range of the λ study. The Overview tab shows one card per method with the PSNR, gain, iterations and time, marks the best and fastest method, and displays the images in the arrangement that makes them largest. The Convergence tab plots the exact energy gap, the PSNR and the relative change per iteration and the PSNR against wall time; the Lambda study tab plots PSNR against λ , the L-curve and the cost; Report lists all numbers and Log records the session. Runs can be stopped and optionally repeat on every change. Charts are exported as PDF or SVG, images as PNG and data as CSV into a time-stamped folder.

V. RESULTS

All experiments were run by a console program that compiles the solver classes of the application without changes, on an Intel Core i5-1345U with 32 GB of memory under Windows 11 (MSVC, Release build). The test images Cameraman, Barbara and Ape have 225×225 pixels. Unless stated otherwise

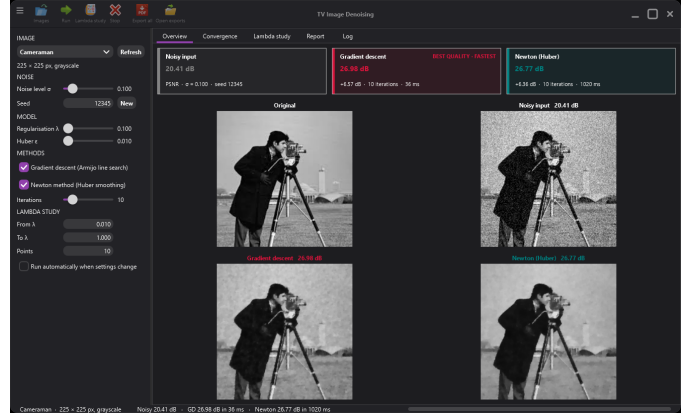


Fig. 1. Overview tab after a comparison on Cameraman with $\sigma = 0.1$, $\lambda = 0.1$, $\varepsilon = 0.01$ and 10 iterations.

$\sigma = 0.1$ with seed 12345, $\lambda = 0.1$, $\varepsilon = 10^{-3}$ for gradient descent (GD) and $\varepsilon = 10^{-2}$ for the Newton-type method (NT). GD ran 300 and NT 30 iterations.

A. Quality and convergence

Fig. 2 shows the results and Table I the numbers. Both methods raise the PSNR by 3.4 to 6.5 dB and produce visually similar images with the piecewise constant regions typical of TV denoising; fine texture such as the stripes on Barbara's scarf and the fur of Ape is partly removed. After the application default of 10 iterations GD is ahead on all three images, by up to 0.44 dB on Ape.

Fig. 3(a) shows that the PSNR peaks early and then decreases. GD reaches its best value after 40 iterations on Cameraman, 6 on Barbara and 4 on Ape; NT after 5, 2 and 2 iterations. With $\lambda = 0.1$ the minimiser smooths more than the PSNR-optimal solution (Section V-C), so iterates that are not yet converged are closer to the clean image than the minimiser itself.

In terms of the objective, Fig. 3(b), GD reaches an energy within 2% of the lowest observed value after 18 to 33 iterations (36 to 136 ms) and within 1% after 47 to 65 iterations. NT needs only 5 to 7 iterations for 2%, but these take 555 to 583 ms. Its energy then levels off 1.7% (Cameraman), 1.0% (Barbara) and 0.9% (Ape) above the GD result, because it minimises F_ε with a ten times larger ε .

The iterates behave differently, as Fig. 4(a) shows. The relative change of NT falls by a factor of ten or more every ten iterations on all images, to 2.1×10^{-6} to 4.0×10^{-6} after 30 iterations, which is the linear convergence expected from Section III. The relative change of GD is not monotone, because the accepted step length changes between iterations, and after 20 iterations it decreases only slowly, from about 10^{-3} to between 3.1×10^{-5} and 1.2×10^{-4} at $k = 300$.

B. Smoothing parameter

Fig. 5 and Fig. 4(b) vary ε for NT. A larger ε makes the weights (7) more uniform and the problem better conditioned: after 25 iterations the relative change is 5.6×10^{-5} to $8.6 \times$

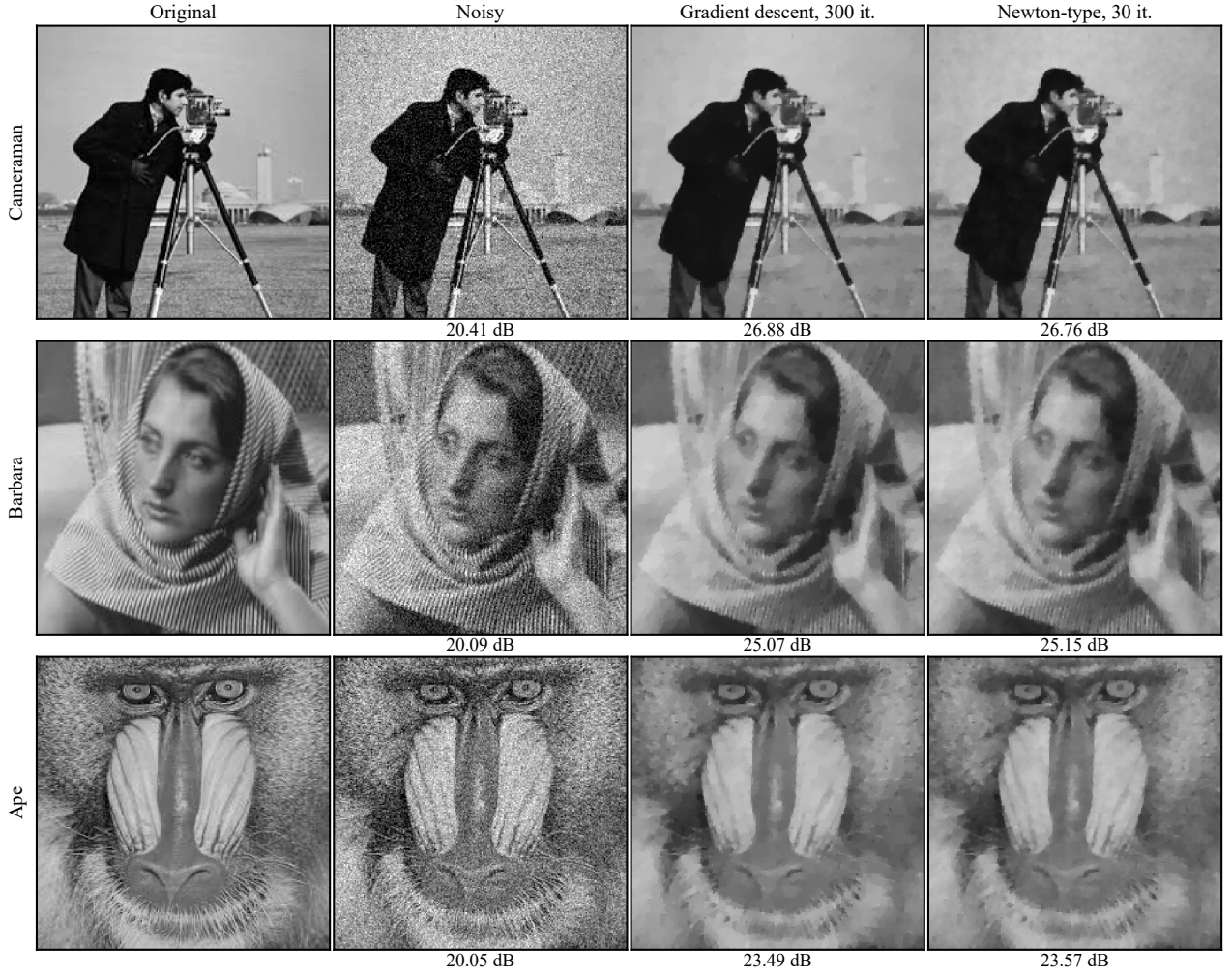


Fig. 2. Test images, noisy input ($\sigma = 0.1$) and results of both methods with $\lambda = 0.1$. Numbers are PSNR values.

TABLE I

COMPARISON AT $\sigma = 0.1$, $\lambda = 0.1$. BEST: HIGHEST PSNR OVER ALL ITERATIONS (ITERATION IN BRACKETS). E : EXACT TV ENERGY OF THE FINAL ITERATE. TIME: MEAN WALL TIME PER ITERATION.

Image	Noisy	Gradient descent, $\varepsilon = 10^{-3}$					Newton-type, $\varepsilon = 10^{-2}$				
		Best	$k = 10$	$k = 300$	E	ms/it.	Best	$k = 10$	$k = 30$	E	ms/it.
Cameraman	20.41	27.04 (40)	26.98	26.88	356.7	3.9	26.79 (5)	26.77	26.76	362.7	100.6
Barbara	20.09	25.66 (6)	25.59	25.07	388.0	3.5	25.72 (2)	25.18	25.15	391.9	122.5
Ape	20.05	24.24 (4)	24.05	23.49	427.3	5.3	24.41 (2)	23.61	23.57	431.1	117.2

10^{-5} for $\varepsilon = 10^{-3}$ but 7.6×10^{-8} to 7.8×10^{-8} for $\varepsilon = 10^{-1}$, where it is limited by the 32-bit pixel values. The price is a larger bias: from $\varepsilon = 10^{-3}$ to 10^{-1} the exact energy of the result increases by 19% on Cameraman, 15% on Barbara and 14% on Ape. The PSNR responds differently per image. On Cameraman it stays at 26.74 to 26.76 dB up to $\varepsilon = 0.02$ and drops to 26.26 dB at 0.1. On the textured images it increases with ε , by 0.74 dB on Barbara and 1.07 dB on Ape, because a large ε penalises small gradients quadratically and removes less texture, which partly compensates for the too large λ .

C. Regularisation weight

Fig. 6 repeats both methods for 15 values of λ . The best value is $\lambda = 0.072$ for Cameraman and Barbara and 0.052 for Ape, all below the default 0.1. At the best λ the methods agree within 0.08 dB: 27.39 and 27.31 dB on Cameraman, 25.80 and 25.83 dB on Barbara, 24.49 and 24.53 dB on Ape for GD and NT. For λ above the optimum the curves separate. At $\lambda = 1$, 200 GD iterations leave the total variation of Cameraman at 752, while NT reaches 344; by (13) the Lipschitz constant grows to 8001, so GD takes ten times shorter steps and is far

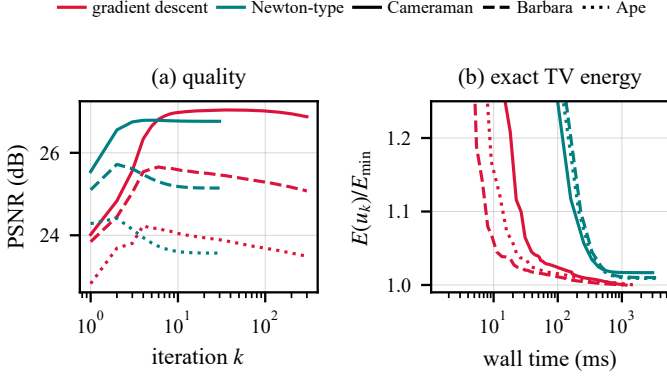


Fig. 3. (a) PSNR per iteration. (b) Exact TV energy against wall time, relative to the lowest energy observed for the image.

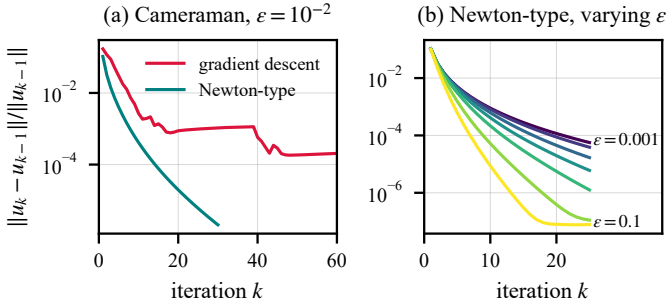


Fig. 4. Relative change $\|u_k - u_{k-1}\|/\|u_{k-1}\|$. (a) Both methods on Cameraman. (b) Newton-type method for $\epsilon \in \{1, 2, 5\} \times 10^{-3}$, $\{1, 2, 5\} \times 10^{-2}$ and 10^{-1} , dark to light.

from converged. Its higher PSNR there is again an effect of early stopping. On the L-curve both methods lie on the same curve.

Table II extends the study to four noise levels. The best λ grows with the noise, from 0.016 to 0.025 at $\sigma = 0.05$ to 0.158 at $\sigma = 0.2$, and both methods select the same value in every case. The PSNR gain grows from about 2.1 to 5.0 dB at $\sigma = 0.05$ to 7.8 to 8.8 dB at $\sigma = 0.2$, and the best PSNR of the two methods differs by at most 0.13 dB.

D. Cost per iteration

Fig. 7 shows the cost of one iteration as the image grows. The dense path follows the $O(N^3)$ cost of LU: from $N = 1024$ to $N = 4096$ the time rises from 31 ms to 1.61 s, an exponent of 2.85. At the switch to the sparse solver the time drops by a factor of 145, to 11 ms at $N = 5184$. From there to $N = 202,500$ the sparse path grows to 495 ms, an observed exponent of 1.04, and GD from 0.24 to 10.4 ms. At every size above the switch one NT iteration costs 20 to 50 GD iterations.

VI. DISCUSSION

For this problem the two methods trade iteration count against cost per iteration. NT needs about ten times fewer iterations to reach a given energy or its best PSNR, and its iterates settle quickly and monotonically, which makes stopping easy. GD is 22 to 35 times cheaper per iteration and

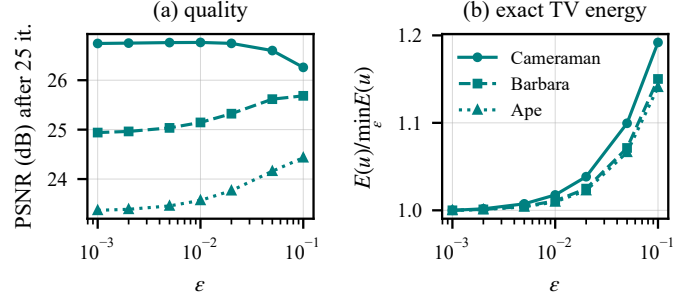


Fig. 5. Newton-type method after 25 iterations for different ϵ . (a) PSNR. (b) Exact TV energy relative to the smallest value over ϵ .

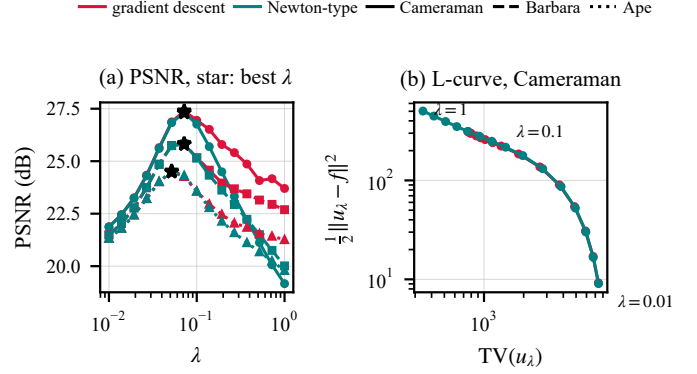


Fig. 6. Study over 15 logarithmically spaced values $\lambda \in [0.01, 1]$ with 200 GD and 20 NT iterations. (a) PSNR, the star marks the best λ . (b) L-curve for Cameraman.

reaches the same quality earlier in wall time at 225×225 pixels. Its progress depends on L in (13): a small ϵ or a large λ slows it down, as the study at $\lambda = 1$ shows.

The smoothing parameter matters more for NT than for GD, because the user can change it and because it trades bias for conditioning. With $\epsilon = 10^{-2}$ the bias is about 1% of the energy and invisible in the images, but at 10^{-1} it is 14 to 19%.

The results also show that the minimiser of the model is not the best denoised image. PSNR peaks after a few iterations at the default λ and the methods only agree closely when λ is chosen well. For a user, a λ study is therefore more useful than running either method to high accuracy, and the application provides one.

The study has limitations. The PSNR-optimal λ uses the clean image, which is unknown in practice. Timings depend on the sparse solver and memory hierarchy of one machine. Methods that avoid smoothing, such as Chambolle's projection algorithm [3], FISTA [5] or the primal-dual algorithm of Chambolle and Pock [4], were not compared.

VII. CONCLUSION

Gradient descent with Armijo backtracking and the lagged diffusivity Newton-type method were implemented in a cross-platform C++ application and compared on identical seeded noise. At the PSNR-optimal λ both give the same image

TABLE II
BEST λ ON AN 11-POINT GRID IN $[0.01, 1]$ AND BEST PSNR (DB) OF GD
AND NT FOR SEVERAL NOISE LEVELS

σ	Cameraman				Barbara				Ape			
	Noisy	λ^*	GD	NT	Noisy	λ^*	GD	NT	Noisy	λ^*	GD	NT
0.05	26.26	0.025	31.22	31.09	26.05	0.025	29.57	29.57	26.04	0.016	28.18	28.18
0.10	20.41	0.063	27.31	27.25	20.09	0.063	25.89	25.91	20.05	0.063	24.45	24.49
0.15	17.07	0.100	24.95	24.99	16.74	0.100	24.20	24.27	16.63	0.100	22.95	23.01
0.20	14.83	0.158	23.41	23.39	14.52	0.158	23.31	23.36	14.35	0.158	22.12	22.18

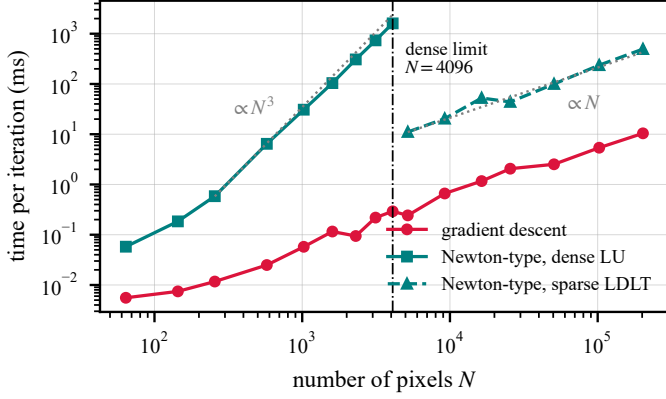


Fig. 7. Wall time per iteration for Cameraman resampled to $n \times n$ pixels, n from 8 to 450. The Newton-type method switches from dense LU to sparse LDL^T above $N = 4096$.

quality within 0.08 dB. The Newton-type method converges in far fewer iterations and decreases the smoothed objective monotonically, but each iteration needs a sparse factorisation that costs 20 to 50 gradient steps and its smoothing adds a bias of about 1% in the exact energy. Gradient descent is faster in wall time for the image sizes studied, but slows down for large λ or small ε . The dense Newton system becomes impractical beyond a few thousand pixels, while the sparse solver scales almost linearly up to 202,500 pixels.

REFERENCES

- [1] L. I. Rudin, S. Osher, and E. Fatemi, “Nonlinear total variation based noise removal algorithms,” *Physica D: Nonlinear Phenomena*, vol. 60, no. 1–4, pp. 259–268, 1992.
- [2] C. R. Vogel and M. E. Oman, “Iterative methods for total variation denoising,” *SIAM Journal on Scientific Computing*, vol. 17, no. 1, pp. 227–238, 1996.
- [3] A. Chambolle, “An algorithm for total variation minimization and applications,” *Journal of Mathematical Imaging and Vision*, vol. 20, no. 1–2, pp. 89–97, 2004.
- [4] A. Chambolle and T. Pock, “A first-order primal-dual algorithm for convex problems with applications to imaging,” *Journal of Mathematical Imaging and Vision*, vol. 40, no. 1, pp. 120–145, 2011.
- [5] A. Beck and M. Teboulle, “Fast gradient-based algorithms for constrained total variation image denoising and deblurring problems,” *IEEE Transactions on Image Processing*, vol. 18, no. 11, pp. 2419–2434, 2009.
- [6] P. J. Huber, “Robust estimation of a location parameter,” *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.
- [7] L. Armijo, “Minimization of functions having Lipschitz continuous first partial derivatives,” *Pacific Journal of Mathematics*, vol. 16, no. 1, pp. 1–3, 1966.
- [8] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.
- [9] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge University Press, 2004.

- [10] T. F. Chan and P. Mulet, “On the convergence of the lagged diffusivity fixed point method in total variation image restoration,” *SIAM Journal on Numerical Analysis*, vol. 36, no. 2, pp. 354–367, 1999.
- [11] T. F. Chan, G. H. Golub, and P. Mulet, “A nonlinear primal-dual method for total variation-based image restoration,” *SIAM Journal on Scientific Computing*, vol. 20, no. 6, pp. 1964–1977, 1999.
- [12] A. George, “Nested dissection of a regular finite element mesh,” *SIAM Journal on Numerical Analysis*, vol. 10, no. 2, pp. 345–363, 1973.
- [13] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2006.
- [14] I. Džafić, “natID: native interface design framework,” <https://github.com/idzafic/natID>, accessed Sep. 2026.